

used to illustrate the Boolean model? Let us modify them a little, so they appear as follows:

```
a.txt: apple, ball, cat
b.txt: Dogs love cats but cats love balls.
c.txt: Cats hate dogs and dogs love eels.
d.txt: dog, eel, fox
```

These example documents will not win any literary prizes, but will do famously to illustrate how this model works. Our term space (ignoring word forms) is as follows:

```
[ apple, ball, cat, dog, eel, fox, hate, love ]
```

Let us draw the term-document incidence matrix, too (Table 1). Note that we introduce term weighting for each document by giving due importance to a term's frequency in a document.

In a search engine, ignoring word forms (example: running, runner) and mapping them to a root word ('run' in this case) is done by a process called stemming. This is essential to keep the vector space from burgeoning out of proportion, and also to make life easier for the search user—a search for 'run' will also bring up documents with 'running' and 'runner', and perhaps improve both precision and recall. We also ignored the words 'and' and 'but'. These are so called stop-words that are part of any document, and muddly up the vector space if allowed to pass through.

Now, if we were to search for the term *dog*, our search query vector would be as follows:

```
[ 0 0 0 1 0 0 0 0 ]
```

...with a magnitude $|Q|$ of 1.

	a.txt	b.txt	c.txt	d.txt
apple	1	0	0	0
ball	1	1	0	0
cat	1	2	1	0
dog	0	1	2	1
eel	0	0	1	1
fox	0	0	0	1
hate	0	0	1	0
love	0	2	1	0
$ D $	1.73	3.16	2.83	1.73

Table 1: Term-document matrix

You might have guessed already that the search query vector is nothing but another document with solely our search terms, and all that the vector space model does is compare the closeness of documents in terms of their vectors. Let us then compute the cosine similarity of our search term vector with the document *b.txt* to illustrate the computation:

$$\cos \theta = (Db.txt.Q) / (|Db.txt| |Q|) = (0*0 + 1*0 + 2*0 + 1*1 + 0*0 + 0*0 + 0*0 + 2*0) / (3.16*1) = 0.32$$

Table 2 shows the cosine values for all documents with respect to our search term vector.

	a.txt	b.txt	c.txt	d.txt
$\cos \theta$	0	0.32	0.7	0.58

Table 2: Cosine values

There are a couple of interesting things we can learn from Table 2. It should not surprise us that *a.txt* scores 0, making it the least relevant (in fact, it should not even make it to the search results); and *c.txt* scores the highest, making it the most relevant according to our model. But what does surprise us is that *d.txt* scores higher than *b.txt*. This shows the predisposition of this model towards shorter documents, term weights being equal. Intuitively, you can appreciate that the term *dog* has a higher contribution towards the resultant document vector in *d.txt* than in *b.txt*. However, using *idf* (inverse document frequency) in addition to *tf* (term frequency) for term weighting can mitigate the situation.

The vector space model is powerful, but is very computation- and memory-intensive. All those vector inner products for a large collection take their toll on performance. On the other hand, comparing documents or finding similar ones (the familiar 'more like this' or 'find similar' hyperlink) is straightforward, given how the model works. Search term vectors can be arbitrarily long because the search terms are treated as yet another document. Although initial search results might be disappointing, this model gives us the advantage of relevance feedback. What this means is, if we find a highly relevant document in the result set, we can use the 'more like this' link to drill down deeper into a subset of the results. When this happens, the significant terms of the highly relevant document are used to reconstitute the search string, which is then resubmitted, giving us a revised and, hopefully, more relevant result set.

Probabilistic model

The probabilistic model uses an approach of estimating the probability of a search query term indexing a relevant document. (A term is said to index a document if that term appears as a key word in that document in the search engine's database.) It is variously termed as 'difficult' and 'ad hoc'. Working with a search engine implementing this model might also need some getting used to. However, it has a sound theoretical basis, which we will barely touch upon here. The so-called BM25 [2] weighting scheme is most popular since it's considered the most suitable.

Briefly, this is how it gets off the ground, as described in Reference [3]. If we were to define some probabilities for a query term *t*, as shown below:

```
p probability that t indexes a relevant document
q probability that t indexes a non-relevant document.
```

...we would consequently get: